

step6_time_test

May 29, 2026

```
[1]: # =====  
# step6_timeline_anchor_remedy.py  
# May 30, 2026  
# =====  
  
from pathlib import Path  
import numpy as np  
import pandas as pd  
from sklearn.cluster import KMeans  
from sklearn.decomposition import PCA  
from sklearn.ensemble import IsolationForest  
from sklearn.feature_selection import mutual_info_classif  
from sklearn.impute import SimpleImputer  
from sklearn.metrics import (  
    average_precision_score,  
    balanced_accuracy_score,  
    f1_score,  
    precision_recall_curve,  
    roc_auc_score,  
    confusion_matrix,  
)  
from sklearn.model_selection import StratifiedKFold  
from sklearn.preprocessing import RobustScaler  
  
RANDOM_STATE = 42  
DATA_PATH = Path('DataSet.csv')  
TARGET_COL = 'F3924'  
ID_COL = 'Unnamed: 0'  
  
# Audited leaky features removed  
LEAKY_FEATURES = ['F3912', 'F2230', 'F3886', 'F3889', 'F3891', 'F3892']  
  
BANK_FEATURES = [  
    'F115', 'F321', 'F527', 'F531', 'F670', 'F1692', 'F2082', 'F2122',  
    'F2582', 'F2678', 'F2737', 'F2956', 'F3043', 'F3836', 'F3887',  
    'F3889', 'F3891', 'F3894',  
]
```

```

PLACEHOLDER_VALUES = {-99999999, 99999999, -9999999, 9999999, -999999, 999999, ↵
↵-9999, 9999}
LARGE_ABS_THRESHOLD = 1e7
PLACEHOLDER_MIN_FRAC = 0.002

TOP_MI = 25
TOP_GAP = 25
N_PCA = 3
N_CLUSTERS = 3
LOW_CARD_MAX_UNIQUE = 12
TARGET_ENCODING_SMOOTHING = 20.0
TEMPORAL_PARSE_MIN_FRAC = 0.7
ANOMALY_N_ESTIMATORS = 200
SMOTE_RATIO = 0.1
BLEND_WEIGHTS = (0.6, 0.4)

# Configurable Cost Ratios
COST_FN_RATIO = 5.0 # Missed fraud is 5x more costly than a false alarm
COST_FP_BASE = 1.0

# Ensure package availability
try:
    import xgboost as xgb
except ImportError:
    xgb = None

try:
    import lightgbm as lgb
except ImportError:
    lgb = None

try:
    from imblearn.over_sampling import SMOTE
    has_imblearn = True
except ImportError:
    SMOTE = None
    has_imblearn = False

# =====
# Helper Functions (From Step 5)
# =====

def detect_placeholder_values(frame: pd.DataFrame, abs_threshold: float, ↵
↵min_frac: float) -> dict:
    placeholder_map = {}
    for col in frame.columns:

```

```

        series = frame[col].dropna()
        if series.empty:
            continue
        extreme = series[series.abs() >= abs_threshold]
        if extreme.empty:
            continue
        counts = extreme.value_counts()
        candidate = counts.index[0]
        if counts.iloc[0] / len(series) >= min_frac:
            placeholder_map[col] = candidate
    return placeholder_map

def identify_temporal_columns(frame: pd.DataFrame, min_frac: float = 0.1,
    TEMPORAL_PARSE_MIN_FRAC, sample_size: int = 5000) -> list:
    temporal_cols = []
    for col in frame.columns:
        series = frame[col].dropna().astype(str)
        if series.empty:
            continue
        if len(series) > sample_size:
            series = series.sample(sample_size, random_state=RANDOM_STATE)
        parsed = pd.to_datetime(series, errors='coerce')
        if parsed.notna().mean() >= min_frac and parsed.nunique(dropna=True) > 1:
            temporal_cols.append(col)
    return temporal_cols

def build_row_stats(frame: pd.DataFrame) -> pd.DataFrame:
    values = frame.to_numpy(dtype=float)
    mask = ~np.isnan(values)
    non_missing = mask.sum(axis=1)
    total = values.shape[1]
    missing_rate = 1.0 - (non_missing / total)
    zero_rate = np.where(non_missing > 0, (values == 0).sum(axis=1) / non_missing, 0)
    positive_rate = np.where(non_missing > 0, (values > 0).sum(axis=1) / non_missing, 0)
    negative_rate = np.where(non_missing > 0, (values < 0).sum(axis=1) / non_missing, 0)

    with np.errstate(all='ignore'):
        mean = np.nanmean(values, axis=1)
        std = np.nanstd(values, axis=1)
        min_val = np.nanmin(values, axis=1)
        max_val = np.nanmax(values, axis=1)
        median = np.nanmedian(values, axis=1)
        q25 = np.nanpercentile(values, 25, axis=1)

```

```

q75 = np.nanpercentile(values, 75, axis=1)
abs_mean = np.nanmean(np.abs(values), axis=1)

iqr = q75 - q25
return pd.DataFrame({
    'row_non_missing_count': non_missing,
    'row_missing_rate': missing_rate,
    'row_zero_rate': zero_rate,
    'row_positive_rate': positive_rate,
    'row_negative_rate': negative_rate,
    'row_mean': mean,
    'row_std': std,
    'row_min': min_val,
    'row_max': max_val,
    'row_median': median,
    'row_q25': q25,
    'row_q75': q75,
    'row_iqr': iqr,
    'row_abs_mean': abs_mean,
}, index=frame.index)

def build_step2_table(numeric_frame: pd.DataFrame, target: pd.Series, row_stats:
    pd.DataFrame) -> tuple:
    bank_features = [col for col in BANK_FEATURES if col in numeric_frame.
        columns and numeric_frame[col].notna().any()]
    mi_candidates = numeric_frame.loc[:, numeric_frame.nunique(dropna=True) > 1]
    top_mi_cols = []
    if mi_candidates.shape[1] > 0:
        mi_imputer = SimpleImputer(strategy='median')
        mi_values = mi_imputer.fit_transform(mi_candidates)
        mi_scores = mutual_info_classif(mi_values, target,
            random_state=RANDOM_STATE)
        mi_series = pd.Series(mi_scores, index=mi_candidates.columns).
            sort_values(ascending=False)
        top_mi_cols = mi_series.head(TOP_MI).index.tolist()

    missing_gap = (numeric_frame.loc[target == 1].isna().mean() - numeric_frame.
        loc[target == 0].isna().mean()).abs().sort_values(ascending=False)
    top_gap_cols = missing_gap.head(TOP_GAP).index.tolist()

    selected_cols = list(set(bank_features + top_mi_cols + top_gap_cols))
    if top_gap_cols:
        missing_flags = numeric_frame[top_gap_cols].isna().astype(int).
            add_prefix('miss_')
    else:
        missing_flags = pd.DataFrame(index=numeric_frame.index)

```

```

compact_frame = pd.concat([numeric_frame[selected_cols], row_stats,
↪missing_flags], axis=1)
compact_frame = compact_frame.loc[:, compact_frame.isna().mean() < 1.0]

metadata = {
    'bank_features': bank_features,
    'top_mi_cols': top_mi_cols,
    'top_gap_cols': top_gap_cols,
    'selected_cols': selected_cols,
}
return compact_frame, metadata

def build_xgb(scale_pos_weight: float):
    return xgb.XGBClassifier(
        n_estimators=500, max_depth=5, learning_rate=0.05,
        subsample=0.8, colsample_bytree=0.8, eval_metric='aucpr',
        scale_pos_weight=scale_pos_weight, tree_method='hist',
        random_state=RANDOM_STATE, n_jobs=-1
    )

def build_lgb(scale_pos_weight: float):
    return lgbl.LGBMClassifier(
        n_estimators=500, learning_rate=0.05, num_leaves=31,
        subsample=0.8, colsample_bytree=0.8, objective='binary',
        scale_pos_weight=scale_pos_weight, random_state=RANDOM_STATE,
        n_jobs=-1, verbosity=-1
    )

def calibrate_probabilities(y_tr, probs_tr, y_va, probs_va):
    from sklearn.calibration import IsotonicRegression
    calibrator = IsotonicRegression(out_of_bounds='clip', y_min=0, y_max=1)
    calibrator.fit(probs_tr, y_tr)
    probs_va_calibrated = calibrator.predict(probs_va)
    return probs_va_calibrated, calibrator

def compute_cost_metrics(y_true, y_pred, cost_fn_ratio, cost_fp):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    cost_fn = cost_fn_ratio * cost_fp
    total_cost = (fn * cost_fn) + (fp * cost_fp)
    cost_per_fraud = total_cost / max((y_true == 1).sum(), 1)
    return {
        'tp': tp, 'tn': tn, 'fp': fp, 'fn': fn,
        'cost_fn': cost_fn, 'cost_fp': cost_fp,
        'total_cost': total_cost, 'cost_per_fraud': cost_per_fraud,
    }

```

```

def find_cost_optimal_threshold(y_true, probs, cost_fn_ratio, cost_fp):
    unique_probs = np.unique(probs)
    min_cost = np.inf
    optimal_thr = 0.5
    for thr in unique_probs:
        y_pred = (probs >= thr).astype(int)
        cost_dict = compute_cost_metrics(y_true, y_pred, cost_fn_ratio, cost_fp)
        if cost_dict['total_cost'] < min_cost:
            min_cost = cost_dict['total_cost']
            optimal_thr = thr
    return optimal_thr, min_cost

# =====
# Step 6 Implementations: Safe Temporal Features
# =====

def encode_categorical_fold(train_raw, valid_raw, train_target,
    categorical_cols):
    train_parts = []
    valid_parts = []
    for col in categorical_cols:
        train_series = train_raw[col].astype('string').fillna('MISSING')
        valid_series = valid_raw[col].astype('string').fillna('MISSING')
        cardinality = train_series.nunique(dropna=False)

        if cardinality <= LOW_CARD_MAX_UNIQUE:
            categories = sorted(train_series.unique().tolist())
            mapping = {value: idx for idx, value in enumerate(categories)}
            train_encoded = train_series.map(mapping).fillna(-1).astype(float)
            valid_encoded = valid_series.map(mapping).fillna(-1).astype(float)
            feature_name = f'{col}_ord'
        else:
            global_mean = float(train_target.mean())
            stats = pd.DataFrame({
                'category': train_series,
                'target': train_target.to_numpy(),
            }).groupby('category')['target'].agg(['mean', 'count'])
            smooth = (stats['count'] * stats['mean'] +
                TARGET_ENCODING_SMOOTHING * global_mean) / (stats['count'] +
                TARGET_ENCODING_SMOOTHING)
            train_encoded = train_series.map(smooth).fillna(global_mean).
            astype(float)
            valid_encoded = valid_series.map(smooth).fillna(global_mean).
            astype(float)
            feature_name = f'{col}_te'

```

```

        train_parts.append(pd.Series(train_encoded, index=train_raw.index,
↳name=feature_name))
        valid_parts.append(pd.Series(valid_encoded, index=valid_raw.index,
↳name=feature_name))

    if train_parts:
        return pd.concat(train_parts, axis=1), pd.concat(valid_parts, axis=1)
    return pd.DataFrame(index=train_raw.index), pd.DataFrame(index=valid_raw.
↳index)

def build_temporal_fold_features_safe(train_raw, valid_raw, temporal_cols,
↳drop_elapsed_days: bool = True):
    """
    Constructs temporal features.
    If drop_elapsed_days=True, drops the absolute continuous elapsed timeline
    metric to eliminate distribution shift in future cohorts.
    """
    train_parts = []
    valid_parts = []
    for col in temporal_cols:
        train_parsed = pd.to_datetime(train_raw[col], errors='coerce')
        valid_parsed = pd.to_datetime(valid_raw[col], errors='coerce')
        base_date = train_parsed.min()
        if pd.isna(base_date):
            base_date = pd.Timestamp('1970-01-01')

        def make_temporal_frame(parsed: pd.Series) -> pd.DataFrame:
            dow = parsed.dt.dayofweek.astype(float)
            month = parsed.dt.month.astype(float)

            features = {
                f'{col}_dow_sin': np.sin(2 * np.pi * dow / 7.0),
                f'{col}_dow_cos': np.cos(2 * np.pi * dow / 7.0),
                f'{col}_month_sin': np.sin(2 * np.pi * (month - 1.0) / 12.0),
                f'{col}_month_cos': np.cos(2 * np.pi * (month - 1.0) / 12.0),
            }
            if not drop_elapsed_days:
                features[f'{col}_elapsed_days'] = (parsed - base_date).dt.
↳total_seconds() / 86400.0

            return pd.DataFrame(features, index=parsed.index)

        train_parts.append(make_temporal_frame(train_parsed))
        valid_parts.append(make_temporal_frame(valid_parsed))

    if train_parts:
        return pd.concat(train_parts, axis=1), pd.concat(valid_parts, axis=1)

```

```

        return pd.DataFrame(index=train_raw.index), pd.DataFrame(index=valid_raw.
↪index)

def add_anomaly_feature(train_frame, valid_frame, train_target):
    imputer = SimpleImputer(strategy='median')
    train_imp = imputer.fit_transform(train_frame) if train_frame.shape[1] > 0
↪else np.zeros((len(train_frame), 1))
    valid_imp = imputer.transform(valid_frame) if valid_frame.shape[1] > 0 else
↪np.zeros((len(valid_frame), 1))

    scaler = RobustScaler()
    train_scaled = scaler.fit_transform(train_imp)
    valid_scaled = scaler.transform(valid_imp)

    normal_mask = (train_target == 0).to_numpy()
    if normal_mask.sum() < 10:
        normal_mask = np.ones(len(train_target), dtype=bool)

    iso = IsolationForest(n_estimators=ANOMALY_N_ESTIMATORS,
↪random_state=RANDOM_STATE, n_jobs=-1)
    iso.fit(train_scaled[normal_mask])

    train_aug = train_frame.copy()
    valid_aug = valid_frame.copy()
    train_aug['anomaly_score'] = -iso.score_samples(train_scaled)
    valid_aug['anomaly_score'] = -iso.score_samples(valid_scaled)
    return train_aug, valid_aug

def augment_step6_fold(base_train, base_valid, raw_train, raw_valid,
↪train_target, categorical_cols, temporal_cols, drop_elapsed_days: bool):
    X_tr = base_train.copy()
    X_va = base_valid.copy()

    cat_tr, cat_va = encode_categorical_fold(raw_train, raw_valid,
↪train_target, categorical_cols)
    temp_tr, temp_va = build_temporal_fold_features_safe(raw_train, raw_valid,
↪temporal_cols, drop_elapsed_days)

    X_tr = pd.concat([X_tr, cat_tr, temp_tr], axis=1)
    X_va = pd.concat([X_va, cat_va, temp_va], axis=1)

    X_tr, X_va = add_anomaly_feature(X_tr, X_va, train_target)
    return X_tr, X_va

# =====
# Execution & Side-by-Side Comparison

```



```

# =====

# 1. Load Data
df = pd.read_csv(DATA_PATH)
if ID_COL in df.columns:
    df = df.drop(columns=[ID_COL])

y = df[TARGET_COL].astype(int)
raw_features = df.drop(columns=[TARGET_COL], errors='ignore')

if LEAKY_FEATURES:
    raw_features = raw_features.drop(columns=[col for col in LEAKY_FEATURES if
    ↪ col in raw_features.columns], errors='ignore')

raw_features = raw_features.replace([np.inf, -np.inf], np.nan)
raw_features = raw_features.replace(list(PLACEHOLDER_VALUES), np.nan)

object_cols = raw_features.select_dtypes(include=['object', 'category']).
    ↪columns.tolist()
temporal_cols = identify_temporal_columns(raw_features[object_cols]) if
    ↪object_cols else []
categorical_cols = [col for col in object_cols if col not in temporal_cols]

numeric_base = raw_features.apply(pd.to_numeric, errors='coerce')
placeholder_map = detect_placeholder_values(numeric_base, LARGE_ABS_THRESHOLD,
    ↪PLACEHOLDER_MIN_FRAC)
for col, value in placeholder_map.items():
    numeric_base[col] = numeric_base[col].replace(value, np.nan)

row_stats = build_row_stats(numeric_base)
step2_table, step2_meta = build_step2_table(numeric_base, y, row_stats)

# Define Folds for Cross Validation
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

def run_evaluation_pipeline(drop_elapsed: bool, label_suffix: str) -> list:
    results = []

    for fold, (train_idx, val_idx) in enumerate(skf.split(step2_table, y),
    ↪start=1):
        X_tr = step2_table.iloc[train_idx].copy()
        X_va = step2_table.iloc[val_idx].copy()
        y_tr = y.iloc[train_idx]
        y_va = y.iloc[val_idx]

        # Apply fold-level augmentations
        X_tr, X_va = augment_step6_fold(

```

```

        X_tr, X_va,
        raw_features.iloc[train_idx], raw_features.iloc[val_idx],
        y_tr, categorical_cols, temporal_cols,
    drop_elapsed_days=drop_elapsed
    )

    # Imputation
    imputer = SimpleImputer(strategy='median')
    X_tr_imp = imputer.fit_transform(X_tr)
    X_va_imp = imputer.transform(X_va)

    # SMOTE
    smote = SMOTE(sampling_strategy=SMOTE_RATIO, random_state=RANDOM_STATE)
    X_tr_res, y_tr_res = smote.fit_resample(X_tr_imp, y_tr)

    spw = (y_tr_res == 0).sum() / max(y_tr_res.sum(), 1)
    xgb_fold = build_xgb(spw).fit(X_tr_res, y_tr_res)
    lgb_fold = build_lgb(spw).fit(X_tr_res, y_tr_res)

    # Blended probs
    probs_tr = (0.6 * xgb_fold.predict_proba(X_tr_imp)[: , 1]) + (0.4 *
    lgb_fold.predict_proba(X_tr_imp)[: , 1])
    probs_va = (0.6 * xgb_fold.predict_proba(X_va_imp)[: , 1]) + (0.4 *
    lgb_fold.predict_proba(X_va_imp)[: , 1])

    # Calibration
    probs_va_calib, _ = calibrate_probabilities(y_tr.values, probs_tr, y_va.
    values, probs_va)

    # Threshold Optimizer
    cost_opt_thr, min_cost = find_cost_optimal_threshold(y_va.values,
    probs_va_calib, COST_FN_RATIO, COST_FP_BASE)
    y_pred = (probs_va_calib >= cost_opt_thr).astype(int)

    fold_pr_auc = average_precision_score(y_va, probs_va_calib)
    fold_macro_f1 = f1_score(y_va, y_pred, average='macro')
    fold_minority_f1 = f1_score(y_va, y_pred, pos_label=1)
    cost_metrics = compute_cost_metrics(y_va.values, y_pred, COST_FN_RATIO,
    COST_FP_BASE)

    results.append({
        'fold': fold,
        'pr_auc': fold_pr_auc,
        'macro_f1': fold_macro_f1,
        'minority_f1': fold_minority_f1,
        'total_cost': cost_metrics['total_cost'],
    })

```

```

        'fn': cost_metrics['fn'],
        'fp': cost_metrics['fp']
    })

    return results

# Run standard anchor-based vs. anchor-free pipelines
results_with_anchor = run_evaluation_pipeline(drop_elapsed=False,
    ↪label_suffix="With Anchor")
results_without_anchor = run_evaluation_pipeline(drop_elapsed=True,
    ↪label_suffix="Anchor Free")

# Summary Comparison
df_with = pd.DataFrame(results_with_anchor)
df_without = pd.DataFrame(results_without_anchor)

comparison = pd.DataFrame({
    'With Anchor Mean': df_with.mean(),
    'With Anchor Std': df_with.std(),
    'Anchor-Free Mean': df_without.mean(),
    'Anchor-Free Std': df_without.std()
}).drop(index='fold')

print("\n=== STEP 6 EVALUATION SUMMARY (Anchor Impact on Stratified K-Fold)
    ↪===")
print(comparison.round(4))

```

```

C:\Users\amart\AppData\Local\Temp\ipykernel_33252\1637318727.py:103:
UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is consistent and
as-expected, please specify a format.
    parsed = pd.to_datetime(series, errors='coerce')
C:\Users\amart\AppData\Local\Temp\ipykernel_33252\1637318727.py:103:
UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is consistent and
as-expected, please specify a format.
    parsed = pd.to_datetime(series, errors='coerce')
C:\Users\amart\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.10_qbz5n
2kfra8p0\LocalCache\local-packages\Python310\site-
packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not
find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by
setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
    warnings.warn(
    File "C:\Users\amart\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.
10_qbz5n2kfra8p0\LocalCache\local-packages\Python310\site-

```

```

packages\joblib\externals\loky\backend\context.py", line 257, in
_count_physical_cores
    cpu_info = subprocess.run(
    File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.10_3.10.3
056.0_x64__qbz5n2kfra8p0\lib\subprocess.py", line 503, in run
    with Popen(*popenargs, **kwargs) as process:
    File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.10_3.10.3
056.0_x64__qbz5n2kfra8p0\lib\subprocess.py", line 971, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.10_3.10.3
056.0_x64__qbz5n2kfra8p0\lib\subprocess.py", line 1456, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,

```

=== STEP 6 EVALUATION SUMMARY (Anchor Impact on Stratified K-Fold) ===

	With Anchor Mean	With Anchor Std	Anchor-Free Mean \
pr_auc	0.8015	0.0735	0.7933
macro_f1	0.8845	0.0466	0.8866
minority_f1	0.7714	0.0921	0.7754
total_cost	18.2000	6.2209	19.0000
fn	2.4000	1.5166	2.8000
fp	6.2000	4.9699	5.0000

	Anchor-Free Std
pr_auc	0.0847
macro_f1	0.0339
minority_f1	0.0671
total_cost	6.6332
fn	1.3038
fp	2.5495